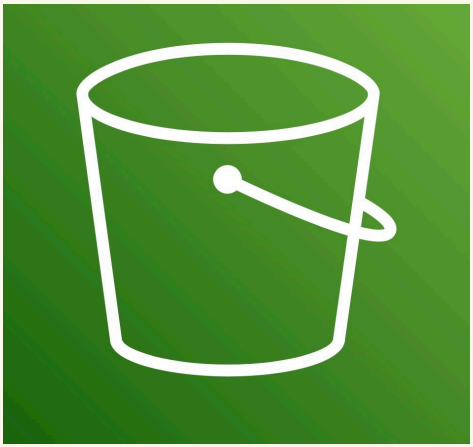
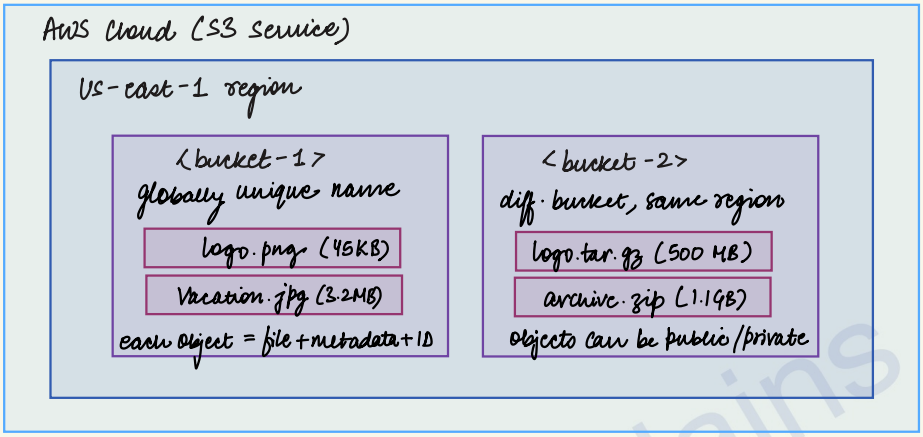
Everything you need to know about -

GENERATING S3 PRESIGN URL USING CLOUDSHELL



S3 Presign URL's and AWS Cloudshell

S3 → giant, internet-connected hard drive in sky → except instead of folders and files, it has buckets & objects



object = file inside bucket → private (only you access them) / public (anyone can)

bucket = container

name must be globally unique

⇒ Problem this lab solves → sharing private files temporarily

we have 3 options

Make file public
↓
anyone on internet can access it forever. No expiry. No control
Security Risk !!
anyone can access your private files

Share Credentials
↓
give them your AWS username + password. Full account access
Catastrophic risk !!
they can delete your entire AWS account

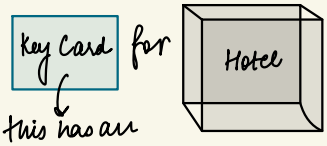
Presigned URL
↓
Time limited, signed link. No credentials shared. Auto-expires. Works in any browser
Right Approach !!
Expires after 60 seconds (or any duration you set)

SurbhiExplains

Presigned URL contains an encrypted signature using your credentials

They are never exposed or shared

is like a

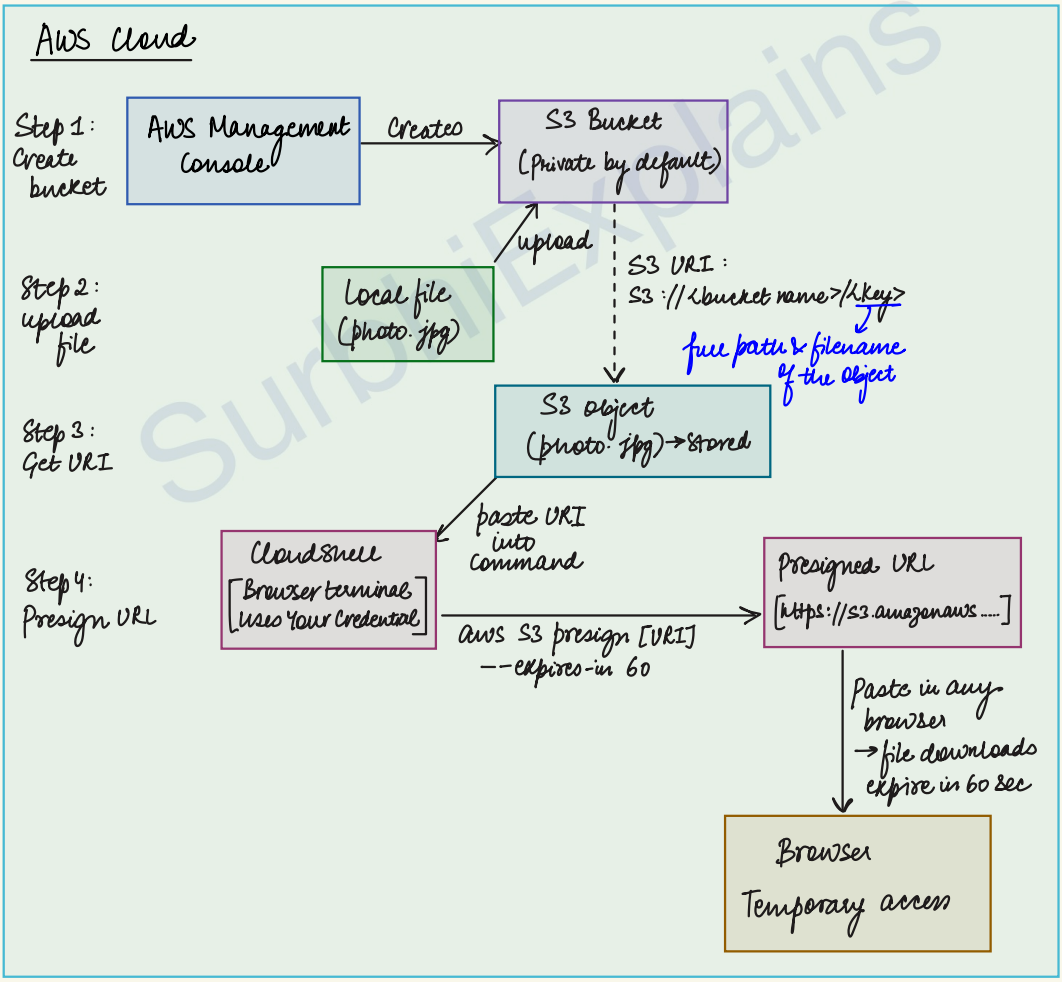


this has an expiry time

→ grants access to 1 specific room → for a limited time ⇒ we dont need to give out master keys

→ After expiration → this becomes completely useless

The full architecture



SurbhiExplains

Presigned URL – under the Hood

https://s3.amazonaws.com/surbhibucket-v1/photo.jpg?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIA...

- Base endpoint → S3.amazonaws.com/bucket/object
- Algorithm → X-AMZ-Algorithm → AWS4-HMAC-SHA256
- Credential scope → WHO signed it (your IAM UserID NOT your password)
- Expiry → X-AMZ-Expires=60 Seconds until death

How AWS verifies the URL when someone visits it

Browser sends URL to S3 → S3 checks
1. Is it expired?
2. Is signature valid?

→ Valid → Serve the file
→ Expired

S3 uses your AWS Secret Key to verify the signature → but never exposes it

Magic is → Cryptographic Signature
↓
AWS uses your secret key to sign the URL parameters
↓

Someone visits your URL
↓
AWS recomputes the signature on its end & checks they match
↓
our actual secret key never leaves AWS
↓
Its just used to produce a signature that can be verified

SurbhiExplains

lab Steps

① Create S3 bucket
- general purpose
- <bucket name>
- object ownership → ACLs disabled → means → Bucket owner owns all objects → Simpler permissions model
- Block public access ✓ → leave it checked
  - default → all public access is blocked
  - → objects are private by default → only YOU can access it

② upload file → get S3 URI → uniform resource identifier

So Basically → you upload object → you click on said object → 2 diff. identifiers

↓

2 ways to identify same object in S3

S3 URI

S3://<bucket-name>/<key>

- used by AWS tools internally
- AWS CLI, SDK, other services understand this format
- can't open in browser

USED IN PRESIGN COMMAND

↓

Copy this → this is used as an input to the presign command

S3 object URL (HTTPS)

https://<bucketname>.s3.amazonaws.com/<key>

This is a browser URL

- Bucket → public → you see object
  - → private → you get access denied error

PRESIGNING CONVERTS THIS TO WORK

③ CloudShell → Honestly → This is AWS's most underrated feature

CloudShell vs traditional AWS Setup

Traditional CLI → our machine

1. Install AWS CLI
2. Run "aws configure"
3. Enter Access key ID & Secret access key
4. Choose region
5. Choose output format

Credentials stored in ~/.aws/ & can be stolen if machine is hacked

Cloudshell → 0 Setup

1. login to AWS console
2. click CloudShell icon [>_]
3. Done. Run commands!!

- Credentials = auto-injected
- AWS CLI pre-installed
- 1GB persistent storage/region
- Amazon Linux 2 environment

SurbhiExplains

Cloudshell inherits your console credentials automatically. when you run aws CLI commands → It already know who you are – it uses your session token to sign URL.

you → password, key } ⊗ No need

The Presign Command dissection

```
aws s3 presign s3://<bucket-name>/<key> --expires-in 60
```

- aws CLI tool
- s3 service
- presign command
- s3 URI of your file
- TTL in seconds

What happens when you paste the URL after 60 seconds? → you get an XML error response from S3.

```
<Error>
  <Code>AccessDenied</Code>
  <Message>Request has expired</Message>
  <X-Amz-Expires>60</X-Amz-Expires>
</Error>
```

Something like this

So S3 is basically telling us → Cryptogenic signature embedded in URL has expired
↓
URL now mathematically invalid
↓
anyone if tries to use it later on will face error

Real world use cases for Presigned URL

1. Secure downloads → user clicks download invoice. Server generates 5-min URL. Link expires after use.
2. Direct browser uploads → PUT presigned URL's allow uploading without going through your server.
3. Video/media streaming → Netflix style: generate a URL per authenticated user session.
4. SaaS file sharing → share with specific people via time limited links
5. CI/CD pipelines → Build artifacts stored in S3. Deploy system & access them via short-lived URL
6. Compliance/audit → Medical records shared for exactly 30 mins (eg.) → full audit trail kept

SurbhiExplains

Core pattern ⇒ Backend Server holds AWS credentials

↓

generates presigned URL

↓

Returns URL to frontend

↓

frontend uses the URL directly → your AWS credentials are NEVER exposed to browser

☆ **extra** → we can also generate presigned PUT URL (for uploads, not just downloads)

↓

Same concept but HTTP method is PUT instead of GET

↓

This is how modern web apps lets user upload pictures directly to S3 → without the data passing through your server

SurbhiExplains